

Ivan Hilkov

Lead AI Engineer — Agent Orchestration, Prompt & Context Engineering

AI-native engineering lead: spec-first, agent-driven, review-gated delivery with Claude Code across 4 organizations · 17 years in software

Contact

strange.mole@gmail.com • linkedin.com/in/ivan-hilkov-a1022154 • t.me/mrbzzz • github.com/ivan-hilckov
Bangkok, Thailand (UTC+7) · remote-first · open to international contracts

Professional Summary

AI engineer who has run software delivery for 4 organizations through Claude Code since autumn 2025: YokeLoop (founder), 3 client engagements and a 5-server pool with self-hosted LLM infrastructure. Recurring prompts became versioned Agent Skills; the skills became a ticket-driven agent orchestrator that moves each change from a product requirements document (PRD) to a plan, agent implementation, a review pass and a ship step. I keep the specification, the acceptance criteria, the deep code reviews and the single confirmation before merge. Guardrails are enforced, not requested: PreToolUse hooks block irreversible operations locally and over SSH, every refusal of the orchestrator's guard is logged, and per-ticket metrics and retrospectives feed back into the prompts. The practice went through 4 generations of homegrown tooling in 7 months and was applied to Electron kiosk software for corporate conference stands, VPN subscription products, a real-time poker client and a Telegram Mini App. 17 years in frontend and full-stack engineering (TypeScript, React, Go, Python), 9 of them leading teams of up to 8 on streaming platforms at 700,000 requests per second and geospatial products, are what make the reviews deep and the specifications precise.

AI Engineering Projects

yokemate — ticket-driven multi-agent orchestrator on Claude Code (YokeLoop, Aug 2026 — Present)

- Context: one engineer delivering tickets for 4 organizations from 3 YouTrack instances and GitHub Issues, on a laptop plus 3 pool development servers.
- What it does: one ordinary Claude Code session at the repository root, opened inside a terminal multiplexer, plus about 35 npm state commands. Each ticket passes a 6-stage lifecycle (new, scouted, planned, running, review, accepted) held in a shared PostgreSQL queue on self-hosted Supabase, so those machines act as equal peers; per-project profiles carry the tracker, the board, the Figma MCP server and the model to use.
- Skills and subagents: 12 engineer-facing skills (/plan, /plan-do, /plan-do-ship, /split, /do, /review, /ship, /journal, /worklog, /note, /retro, /warmup), 6 worker skills spawned into panes, 9 subagents (plan-scout, plan-writer, ticket-writer, task-executor, task-investigator, task-reviewer, retro-journal, retro-prompts, retro-traces). /do is a gated pipeline: tests written before code where a suite exists, the project's own checks, a review pass by a separate subagent, a simplification pass, a documentation sync, one PR per affected repository, CI polling and merge-policy detection before /ship.
- Governance: least-privilege tool access per mode through a PreToolUse guard over Bash, file edits and browser MCP calls (research panes cannot write code or move queue state); permission is asked once per ticket, before the merge; a session marker on every reply exposes an answer from the wrong session; read-only subagents run on a cheaper model tier; an operating CLAUDE.md codifies the reply contract ("what was named is the whole scope", "the user's decisions do not become rules").
- Measured from its own journal and transcripts (Aug 15 to Sep 17, 2026): 125 tickets from plan to merge at a median of 14 hours (75th percentile 28 hours); 92 of 203 tickets (45%) with no human intervention; a PR link on 188 of 188 execution reports; agent cost per ticket at a median of \$10 and a 75th percentile of \$18 over 87 tickets with recorded cost; about \$3,200 of tracked API spend for the month, half of it in sessions outside any ticket. 977 commits on the default branch between 2026-08-15 and 2026-09-14, 64 TypeScript source files, 289 automated test cases, v2.0.0; /retro reads journals, transcripts and guard traces and records accepted or declined prompt changes.

yoke — public Claude Code plugin and skills marketplace, and its lineage (Feb — Jul 2026; the lineage continues in yokemate)

- Context: an off-the-shelf agent workflow produced 300-line plans and 100-file diffs that nobody could review; the answer was a smaller, gated loop with explicit interview and review steps.
- What it does: 14 commands for the full delivery loop (/bootstrap, /grill, /prd, /do, /review, /pr, /merge, /journal, /handoff and others) and the .yoke/ artifact convention (glossary, architecture decision records (ADRs), per-task PRD, plan and report, session journal, handoff files restored by a separate /wake up skill); 258 commits, release 2.0.1 with a changelog, ported to the Pi runtime as yoke-pi.
- Lineage: sp (a homegrown skills pack, 90 commits, Feb to Apr 2026), then yoke, then yokemate-cc (216 commits, Jul to Aug 2026: a fork of the open-source Firstmate piloted on top of the yoke flow in an isolated home directory, ADR-0002; a captain-and-crew harness where one agent dispatches and supervises a crew in multiplexer windows, each on a disposable git worktree, with event-driven supervision that spends no tokens while idle), then yokemate, then piops and

northstar (Pi runtime, Sep 2026). Ancestor codebases are kept read-only; decisions are reimplemented, code is not copied.

- Applied: committed `.yoke/` trails in client repositories of 2 organizations; CLAUDE.md in 28 of 39 working clones and AGENTS.md in 6.

Guardrails for autonomous agent sessions (*Aug — Sep 2026*)

- Context: task tabs run without interactive permission prompts; what used to hold on discipline had to hold on hooks.
- Bash PreToolUse hook with a POSIX ERE denylist of irreversible operations (firewall disable or reset, docker prune with volumes, compose down with volumes, recursive deletion of system roots, filesystem formatting, raw disk writes, stopping sshd, tailscaled or docker, user deletion, overwriting sshd_config) that also catches the same commands wrapped in ssh host `'...'`, so one guard on the workstation covers all 5 pool servers; block and pass fixture tests.
- In the orchestrator every refusal names the allowed alternative and appends one JSON line to an audit log (228 entries in the first 10 days of logging, 2026-09-07 to 09-16: 202 denials, 26 confirmations; the most frequent rules block untracked launches, waiting on a pane and hidden commits), read by `metrics --guard` and `/retro`. Damage is bounded to the ticket's disposable worktree, so the guard fails open on its own errors instead of halting the pipeline.

Self-hosted AI platform on a five-server pool (*YokeLoop, May 2026 — Present*)

- Context: 4 organizations' agents, chats and data on infrastructure under my own control, reachable only over the tailnet.
- Fleet: 5 machines (Ubuntu 22.04/24.04, Arch, Debian 12; Singapore, US, Netherlands) on a Tailscale mesh, SSH on a non-default port with a second door over Tailscale SSH on 4 of the 5, a machine-readable inventory with groups and reboot/upgrade policies, per-machine profiles for agents, and 4 pool-administration skills (snapshot first, both doors before touching access, one machine then the rest, machine policy over the request).
- Services: a template plus 14 `svc-*` Dokploy blueprints kept as git repositories, 10 of them running on the pool (ADRs in the Supabase, MinIO and pgvector stacks): a LiteLLM gateway (one OpenAI-compatible API over cloud and local models with per-client virtual keys, admin over the tailnet only), Open WebUI with SearXNG web search and voice input, a GPU box (Whisper, embeddings, ComfyUI and an OpenAI-images adapter with VRAM budgeting) behind LiteLLM, self-hosted Supabase (13 services), MinIO with the OpenMaxIO object browser built from source, pgvector, Mattermost, n8n, Vaultwarden, Uptime Kuma, a token-usage badge service (Bun and Hono), static sites. Traefik with Let's Encrypt and a DOCKER-USER firewall that accepts ports 80 and 443 from Cloudflare only.
- Monitoring: Uptime Kuma with 23 checks alerting to Mattermost, plus a weekly diff-based fleet report (updates, failed units, disk, kernel, dotfiles drift, reboot pending, unreachable) from a systemd user timer.

hermes-orchestration — B2B hosting of isolated AI agents on bare metal (*YokeLoop, Aug 2026*)

- Provisioning layer of a partner-designed prototype platform for hosting isolated AI agents configured for a client's business processes on a dedicated EPYC server: provisioning service with presets, LiteLLM virtual-key issuance, a provisioner frontend, credential-rotation runbook, gateway smoke tests; provisioner running on the pool.

Knowledge and continuity layer

- Journals as agent memory: pool-wide monthly journal with 239 entries in 2 months, per-repo `.yoke/journal.md`, `/journal` writing a handoff and `/wakeup` restoring it with a repository-drift check; 20.5% of the 4,076 commits analyzed locally update a journal, handoff or worklog.
- Writing quality is toolled: a shared writing rule included by every skill, a Vale vocabulary with a pre-commit check on knowledge and journal files, and a user-level plain-language rule for anything a human reads.
- 9 MCP servers in daily use (3 YouTrack instances, 2 Figma, 2 Dokploy, SSH to pool machines, PostgreSQL) and 9 Claude Code plugins; a custom status line and a shell shim that keeps agent flags on multiplexer session restore.

Professional Experience

YokeLoop — Founder, AI Engineering Lead — Apr 2026 — Present (tooling line since Feb 2026) — remote

Own company (GitHub organization created April 2026 around the yoke plugin, whose history starts February 2026): "We harness LLM agents to build SaaS for small and mid-size businesses. Open-source first." Sole or main engineer; a second contributor sends small PRs. From February 2026 the client work below was delivered through the tooling described above.

- Authored yoke, a public Claude Code plugin marketplace for the full delivery loop (258 commits, release 2.0.1, `.yoke/` artifact convention) and its Pi port yoke-pi (0.0.1–0.0.3).
- Piloted the captain-and-crew model in yokemate-cc, a fork of the open-source Firstmate (216 commits, an ADR-recorded pilot in an isolated home directory) and folded the lessons into yokemate.
- Designed and built yokemate, the orchestrator described above: typed commands, tickets from 3 YouTrack instances and GitHub Issues, project profiles with boards, a shared Postgres work queue with transactional moves, a window table for multiplexer panes, hooks on PreToolUse, SessionStart (git sync, single main chat lock, warm-up digest) and UserPromptSubmit (inbox); 977 commits in one month, 289 test cases, v2.0.0; per-ticket metrics above.
- Began porting the flow to a second agent runtime (Pi): piops (Pi-based orchestration chat with SQLite state and per-mode panes, 27 commits) and northstar (requirements for the next control agent).

- Designed the service blueprint system (Traefik-only, no host ports, pinned images, secrets only in the deployment platform) and stamped 14 svc-* blueprints from it.
- Runs the GPU inference box (Whisper, embeddings, ComfyUI) behind LiteLLM with a pinned driver and tailnet-only access; sized VRAM reservations from measured usage.
- Built the provisioning layer of hermes-orchestration, the partner-designed B2B agent-hosting platform (provisioner service, LiteLLM virtual keys, credential rotation runbook, smoke tests).
- Shipped nook, a Telegram Mini App social network for city place lovers, as sole developer: feed, place sheets, publish flow, badges and share cards, weekly city challenges, motion vocabulary with reduced-motion support, lazy images and chunked screens with a bundle-size floor; 3 deployable apps in a pnpm monorepo, 52 test files with a full-flow run in CI against a real PostgreSQL database, prebuilt images on GitHub Container Registry deployed with Dokploy; 213 commits, 52 PRs.
- Operates the 5-server pool: inventory, machine profiles, policies, tailnet-only lockdown, the Bash PreToolUse guard on the workstation covering SSH commands to every machine, Uptime Kuma, the weekly Mattermost report; server dotfiles with a gitleaks scan; an organization-wide reusable GitHub Actions workflow for Telegram notifications.
- Numbers: about 1,630 commits in 29 of the organization's 30 repositories, 189 PRs; reviews happen inside the agent pipeline (ship reports, re-review rounds) rather than in the GitHub UI.

Talking Birds & Flying Fish (TB-FF) — Lead Engineer, Electron kiosk platform — Oct 2025 — Present — remote

Production studio building interactive installations: Electron kiosk apps for the conference stands of a global technology vendor, Unreal Engine installations and museum exhibits (client engagement; end customers under NDA). Organization admin; sole or majority author on the platform repositories, 40 to 80% on the installation apps; 3 to 5 people per repository.

- Built and maintained the studio's Electron kiosk template (196 of 294 commits, Oct 2025 — Jun 2026): auto-update via electron-updater, release flow with a published-version gate and Telegram release notifications, smoke-build workflow, architecture and bootstrapping docs; every later event app is stamped from it.
- Co-authored the shared React library @tb-ff/web-toolkit (111 of 155 commits): 9-grid video and image slide container, password-protected settings screen, IndexedDB media cache; 5 releases from 2.4.3 to 2.5.1 in June 2026 fixing video cross-fade and buffer teardown.
- Built a product-demo kiosk for an enterprise AI assistant (374 commits, top contributor): 5 demo scenarios, language persistence, Japanese translation, statistics export logging.
- Designed a universal statistics template and integrated it into four event apps, including timezone handling and SQLite migrations.
- Delivered a security vendor's game arena at a major US industry conference (March 2026, 5 player stations on site): the player-station app (235 commits; 27-screen flow, 3 timed touch games, dual display per station, offline leaderboard buffer with SQLite dual-write and batch sync to Supabase), the live leaderboard display (Supabase polling, auto-update overlay, a portrait variant for a second conference), a moderation panel with Supabase Auth built in one day, and a research spike on a two-process Electron app with WebSocket drag transfer.
- Built a conference kiosk for an AI-for-research event (115 commits): a 4-chapter video carousel on a portrait display driven by an Arduino serial encoder, scroll event-loss fixes, calibration overlay.
- Architected a two-PC, server-authoritative room simulator for a conference installation as 3 repositories: server (253 of 324 commits; Express and WebSocket state owner, Arduino buttons and lights, voiceover and music timing), client (122 of 182; renderer for 4 screens forwarding 4 buttons) and a shared typed LAN protocol package (0.1 to 0.12.0); both apps released at v2.0.6; about 30 ticket folders of plans and review reports in .yoke/ai/.
- Contributed to a museum's children's exhibit of 8 Electron installations (third of 4 authors, 157 commits): built the Playwright runtime, an Electron launch fixture, visitor walkthroughs for 5 installations and 4 load and endurance tests, an error screen with retry, Git LFS media.
- Built the delivery-robot kiosk for the same museum (118 of 242 commits): control and big-screen apps over WebSocket, a PixiJS robot, webcam selfie to QR, COM-port robot stop; start-gate and serial reconnect fixes, single instance per role; 40 PR merges.
- Added analytics export with city normalization and a drop-off-by-step metric to an AI-maturity quiz kiosk.
- Stack: Electron 37–40, React 18/19, TypeScript 5.9, Vite (electron-vite), Zustand, CSS Modules, PixiJS and Three.js, WebSocket between processes, serialport, SQLite offline buffer synced to Supabase, Sentry/GlitchTip, electron-log with Graylog, electron-builder, GitHub Actions release with Telegram notify, Playwright, pnpm and mise; shared packages on GitHub Packages.
- Ran delivery through the agent pipeline: CLAUDE.md in every repository, .claude/skills (build-with-agent-team, pr-review), .yoke/ ADRs, plans, execution and review reports. About 1,990 commits in 22 repositories, 313 PRs opened, 37 reviewed.

Velvet VPN (velvetnet) — Lead Frontend Engineer, AI-driven delivery — Apr 2026 — Present — remote

VPN subscription business (client engagement): Telegram bot with payments, subscriber portal, public subscription page, UI kit, payment gateway, CRM automation with LLM replies, eSIM platform. Organization member; backend, infrastructure and CRM owned by others; lead engineer for the public subscription surfaces plus a browser-extension R&D track.

- Created @velvetnet/vpn-ui-kit from scratch (353 commits, 1.0 to 1.44.0): per-component entry points and CSS injection via a custom Rollup/Vite setup, i18n bundles, a Setup module rendering per-platform install guides from markdown (remark-directive) with lightbox and deep links; 2 parallel release lines documented in CLAUDE.md; consumed by 3 apps.
- Built the public subscription page from an empty repository (640 commits, 110 PR merges): a link-opened, no-auth page that also runs as a Telegram Mini App; server-side bootstrap injection from the bot, API-driven setup section, white-label mode for resellers, traffic warnings, Sentry noise filtering, build id baked into the page, CI with lint, format and build on every PR, Playwright end-to-end tests against offline and live environments.
- Co-developed the subscriber portal (248 commits, 41 PR merges): saved cards behind a feature flag, API-driven FAQ and help renderer, "About service" pages, subscription URL from key links, Sentry hardening and 401 sign-out, API-driven setup guides on one route; wrote the 20-file docs set and .claude/rules/.
- Prototyped a Chrome Manifest V3 VPN extension (87 commits, 0.4.2): PAC-based proxying with password auto-answer, stable device identity, email-code login, node list and routing rules from a config endpoint, event reporting, seven-screen React popup, release zip via GitHub Actions; wrote the concept, publishing, debugging and competitor-analysis docs.
- Built its research backend in the personal GitHub account (81 commits): Fastify 5 with SQLite, email login, device registration, node configs, admin UI, landing and download page, Let's Encrypt via Dokploy on a pool server.
- Prototyped an evaluation loop for the support CRM's LLM replies: retrieval-augmented generation (RAG) over the knowledge base with pgvector behind a feature flag and a dedicated embedding endpoint, and a promptfoo-based prompt-quality endpoint with an LLM-as-judge; migrated the repo to npm and mise; wrote the LLM interaction docs.
- Cross-service items: Yandex Metrika goals and a low-traffic API field in the bot, language carry-over between apps in the payment front, a Cypress happy-path suite on permanent test subscriptions.
- Stack: React 19, Vite 7/8, TypeScript strict, React Router 7, TanStack Query 5, CSS Modules, i18next, Framer Motion, Sentry, Yandex Metrika, Telegram Mini App bridge, PWA (Workbox), private npm package on GitHub Packages, Docker multi-stage to nginx, GitHub Actions to GHCR to Kubernetes rollout, Playwright and Cypress.
- Numbers: about 1,370 commits in 8 organization repositories plus 81 in the backend; 228 PRs opened, 7 reviewed; 44 minor releases of the UI kit; an end-to-end-only test strategy (Playwright and Cypress suites) by documented decision. CLAUDE.md in every owned repository; AGENTS.md in 2; committed .yoke/ trails in the UI kit and the portal.

PokerNode — Frontend Lead — Dec 2025 — Aug 2026 — remote

Partner studio's React and TypeScript poker client with real-time gameplay (WebSocket and EventSource), Fastify API on Bun with MongoDB, Redis and a gRPC engine, blackjack rooms, crypto deposits (Solana), a prerendered landing selling 5 product units. Team of 2 to 3.

- Led the frontend of the poker client: 377 of 774 commits, 57 PR merges; React 19, Radix UI, TanStack, Fastify prerender, Solana web3.js.
- Built the history lobby and replay table viewer, crypto logos, mobile tables readable at 360 to 414 px.
- Delivered table-engine visuals in 6 staged PRs: even seat ring and wager ring, betting-round boundaries, payouts and side pots in state, dealer button as a moving object, pot collection and payout flight.
- Added SEO: indexable /ru URLs with hreflang, unified titles, og image, sitemap; landing v2 with scenarios, product units, real screenshots captured by a browser script, request-demo form.
- Read poker room configuration from the Rails admin; delivered a blackjack WebSocket server with gRPC handlers, session manager and JWT auth; added replay-history endpoints to the API.
- About 397 commits, 69 PRs opened; CLAUDE.md and AGENTS.md in the repository.

ALMATICA — Lead Developer & Tech Lead — Aug 2025 — Sep 2025 — Moscow

Collective intelligence platform connecting humans, AI agents and organizations for complex problem-solving.

- Audited 3 legacy projects (React/MUI frontend, Go/Fiber backend, Python/FastAPI ML services) and delivered a microservices MVP in 1 month as a monorepo: Go 1.25 API with Fiber, GORM, PostgreSQL 16, Redis 7, JWT with Kinde Auth and RBAC, OpenAPI docs; React 18 with TypeScript, Vite, Tailwind CSS, shadcn/ui, React Hook Form with Zod, orval-generated client.
- Integrated AI services: FastAPI, LangGraph workflows, LangChain, CrewAI multi-agent systems, OpenAI and Anthropic APIs.
- Led 3 engineers; set up sprint planning, grooming, code reviews, CI/CD, Docker Compose environments and a sandbox deployment for stakeholder demos; wrote the roadmap and technical documentation.

Bot Garden — Founder — Jun 2025 — Sep 2025 — remote

AI-powered Telegram bot platform startup.

- Built 4 LLM-backed bot prototypes in 2 months on the OpenAI, Perplexity and Anthropic APIs: hello-bot (Python/aiogram with PostgreSQL and GitHub Actions deploy, v3.0.0), hello-ai-bot, english-teacher-bot (grammar correction, Redis sessions), shawarma-bot (TypeScript food-ordering bot with Fastify API and mini app, 547 tests, 77% coverage).
- Built botgarden-core, a shared multi-bot VPS runtime (Docker, nginx, SSL) with automated deployment; integrated the WATA.pro payment system.

HRONIKA — Co-Founder / Full-stack Engineer — Apr 2025 — Sep 2025 — Moscow

Autonomous timelapse system for construction and infrastructure monitoring.

- Built the prototype on Raspberry Pi 4 with a Canon 1100D, LTE, UPS HAT and Li-ion in an IP67 enclosure, solar-ready.
- Frontend in React, HeroUI, Tailwind CSS (camera dashboard, photo viewing, timelapse generation) with Cypress e2e; backend in FastAPI with Celery and Redis task queues.
- Authored technical documentation and a public roadmap; submitted the application to the Sber500 accelerator. Site: hronika.tech

Tvigle — Lead Frontend Engineer — May 2024 — Oct 2024 — Moscow

Online cinema.

- Maintained the video player (JavaScript, jQuery); integrated ad SDKs (Yandex Ads, VK Ads) via MRAID; refactored the ad module from VAST 2.0 to VAST 4.0.
- Adapted the cinema layout for mobile; built a React Native mobile app (Android and iOS); reviewed and audited the main Next.js project.

START — Lead Frontend Engineer — Jan 2023 — May 2024 — Moscow

Streaming and media; Smart TV and web platform serving 5M+ users.

- Optimized user queries and caching for peak events at 700,000 requests per second, cutting API P95 latency by about 40%.
- Built “Movies on TV” and “TV Series” sections in Next.js with player and TV channel storefront integration; shipped the TV section with program schedules and catch-up TV.
- Refactored channel pages, the FAQ and support flow, login and subscription checkout; extracted the monolithic payment form’s state into Redux (payment-module code halved); streamlined the A/B testing system.
- Maintained the admin editor (React, Bootstrap), server-side Open Graph metadata, lazy-loaded home previews, infinite-scroll feeds and a REST-fed banner grid.
- Led and mentored 4 frontend engineers: estimation, architecture, onboarding, daily syncs. Link: start.ru

Skyeer — Lead Frontend Engineer — Apr 2017 — Jan 2023 — Moscow

Web-GIS platform for UAV data analytics, photogrammetry automation and domain solutions (construction, mining, logistics, landfills).

- UAV Data Analytics Web Service (map.skyeemap.com): delivered the MVP in 3 months for the Gazprom “Unmanned Biathlon” (1st place, accuracy ± 0.02 m³) and evolved it into the full product on React, Redux-Saga, React Router, Mapbox, Cesium, Potree, Recharts, TurfJS, proj4js, GDAL, Django REST Framework, tus.io and Puppeteer.
- Resumable uploads up to 100 GB (tus.io) for GeoTIFF, ECW, DXF, WMS, XYZ, Cesium Ion 3D, point clouds, SHP, KML and GeoJSON; layer comparison and swipe, survey date switching, basemap and projection selection, 360° panoramas (Pannellum), point cloud visualization (Potree), CesiumJS 3D with parity to Mapbox.
- Measurement tools (area, volume, slope, longitudinal profile, elevation points, markers) with draw and import for GeoJSON, WKT, KML, SHP; XLSX and PDF export; attachments; role-based access (Project, Flight, Measurement).
- Authored a 50+ component UI library with a React Cosmos playground; i18next (English, German); Prettier and ESLint; 150+ Jest tests with ~80% business-logic coverage; CI/CD with automated frontend testing and releases.
- Co-implemented Selenium testing with QA (800+ automated tests); release stability improved by ~80%.
- Hired 8 engineers (middle to senior), onboarded and mentored them in a complex GIS domain; established weekly sprints, standups, grooming, rolling releases, demos and retrospectives; organized the documentation process (doc.map.skyeemap.com).
- Automated Photogrammetry (attractor.aero): UI on shared components, upload of 10,000+ photos via tus.io with bandwidth detection and parallel resumable transfers, EXIF display, API requirements and output specs, Selenium tests.
- Quarry Road Monitoring for NLMK: prototype in 2 months on Django, DRF, jQuery, Bootstrap and Leaflet; client coordination, technical supervision, user docs (Habr article).
- Linear Object Monitoring (aeroinpector.ru) and Waste Landfill Monitoring (reo.ru/flyby): architecture advice for video and 3D geodata, video-to-GPS synchronisation prototype, team formation, support from prototype to release.

Amediateka.ru — Senior Frontend Engineer — Feb 2019 — Dec 2019 — Moscow

- SVOD platform: account support sections (FAQ, feedback), payment form and integrations in React/Redux; catalog pages on Next.js with SSR, content cards, episode switcher, player features (actor info, auto-next, recommendations); Open Graph metadata; code splitting.

Ivideon (Cloud Video Surveillance) — Senior Frontend Engineer — Sep 2016 — Apr 2017 — Moscow

- Extended the UI/UX library with a custom timeline and storyboard with event markers, adapted for mobile; evolved a proprietary Redux-like state layer with async loading; built an internal Canvas tool to annotate queues for ML training (automatic visitor counting).

REKOD (JSC NPK REKOD) — Senior Frontend Engineer — Feb 2016 — Sep 2016 — Moscow

- SPA for WWII monument inspections (Create React App, React Router, Redux, Redux-Saga) with a Leaflet cartography module, photo upload with on-image annotations, a React Native app with offline mode and sync, and a white-label architecture with object schemas served over REST.

Liga Stavok — Frontend Developer — Sep 2015 — Jan 2016 — Russia

- UI library of 40+ React components with a custom playground; LESS with dynamic variables; Redux with dynamic slice import; ASP.NET REST integration via Redux Thunk; migrated the build from Gulp to Webpack (20 minutes to 1).

OneTwoTrip — Frontend Developer — May 2015 — Sep 2015 — Moscow

- Maintained a 5,000+ LOC jQuery payment module and refactored it to React with Webpack and Redux; PDF tickets with PhantomJS; upsell and cross-sell popups; ticket search optimized with static data preloading via Lua; email campaign builder UI; mobile-first search page.

Sovzond — Frontend Developer — Jun 2013 — May 2015 — Moscow

- Designed and built the “Satellite Imagery Archive” SPA (BackboneJS, Bootstrap, Leaflet): REST-driven search, polygon drawing, WMS cadastral layers, cart and account for ordering and download, virtualized tables for large datasets.
- Migrated the “Satellite Imagery Catalog” from ExtJS to BackboneJS with white-label theming and geographic restrictions; integrated map components into Django Admin; Grunt and Bower builds; introduced Mercurial and Bitbucket.
- Initiated regression testing: wrote test cases, hired and onboarded the first tester.

undev.ru — Frontend Developer — Jun 2012 — Jun 2013 — Moscow

- BackboneJS SPA for live broadcasts from 1,000+ polling stations (Elections 2012) with Yandex.Maps clustering; a CoffeeScript video player; an archive browser with time and station search; MochaJS/SinonJS tests, Grunt and Bower builds, RequireJS dependency injection.

bigbuzzy.ru — Frontend Developer — Mar 2011 — Jun 2012 — Moscow

- Interfaces in HTML, CSS, jQuery, BackboneJS and XSLT with pixel-perfect cross-browser layout down to IE 6–8; integration into the proprietary Imprimatur engine; UI guidelines and a CSS component library; mentored junior specialists; introduced Git.

JV Media (snob.ru) — HTML/CSS/JS Developer — Aug 2010 — Mar 2011 — Moscow

- Pixel-perfect cross-browser layout integrated into PHP Zend Framework; jQuery plugins (form validation, image viewer); AJAX comment feed; SVN workflows.

PlatZkart LLC — Content Manager / Frontend Developer — Nov 2009 — Jul 2010 — Krasnodar

- Cross-browser layouts integrated into 1C-Bitrix; content management and basic SEO; a Tilda-like site builder; client site maintenance.

Contractstroy LLC — System Administrator — Jan 2008 — Aug 2009 — Krasnodar

- Administered the corporate file server and an Ubuntu mail server; traffic control and load balancing across two providers; supported ~40 workstations, 1C:Enterprise 7.7–8.1 and the corporate website.

Technical Skills

- **Agent platform & prompt engineering:** Claude Code (Agent Skills with SKILL.md, hooks: PreToolUse, SessionStart, UserPromptSubmit; subagents; plugins and marketplaces; CLAUDE.md, AGENTS.md, .claude/rules), MCP (Model Context Protocol) servers, context engineering, prompt engineering, spec-driven development, orchestrator-workers pattern, reviewer and rework loops, model routing by role, Codex CLI, Pi agent runtime, GitHub Copilot
- **Evals, guardrails & observability:** PreToolUse guards with logged refusals, least-privilege tool surfaces per mode, one-confirmation-per-ticket policy, session markers on replies, promptfoo with LLM-as-judge, pgvector retrieval, pipeline metrics (stage medians, interventions), LLM-driven retrospectives over journals and guard traces, Uptime Kuma, Sentry, vale writing checks in pre-commit

- **LLM infrastructure:** LiteLLM (OpenAI-compatible gateway, virtual keys), Open WebUI, SearXNG, GPU inference (Whisper, embeddings, ComfyUI with VRAM budgeting), pgvector, Supabase (self-hosted), MinIO, Dokploy, Docker Compose, Traefik, Let's Encrypt, Cloudflare-only firewalling, Tailscale, n8n, Vaultwarden, Mattermost
- **Delivery tooling:** YouTrack (REST and MCP, three instances), GitHub CLI and Actions, GitHub Packages, Playwright, Cypress, herdr and tmux, git worktrees, GNU Stow, systemd timers, Mattermost and Telegram webhooks, Figma MCP
- **Product stack:** TypeScript, React 18/19, Electron, Vite, Next.js, Telegram Mini Apps, Chrome Manifest V3, Fastify, Node.js, Bun, Python (FastAPI, aiogram), Go (Fiber, GORM), PostgreSQL, SQLite, Redis, WebSocket, gRPC, serial hardware (Arduino)
- **Legacy depth:** geospatial (Cesium, Mapbox, Leaflet, Potree), media streaming (HLS, FFmpeg, VAST ad SDKs), React Native, Redux, QA automation (Selenium, Jest), team leadership (up to 8 engineers)
- **Languages:** Russian (native), English (professional working proficiency)

Education

Bachelor of Science in Theoretical Physics — Kuban State University, 2010

Featured Links

- yokeloop/yoke — Claude Code plugin and marketplace of skills and commands for the full dev loop: github.com/yokeloop/yoke
- projectory-com/sp — the predecessor skills pack (Feb to Apr 2026): github.com/projectory-com/sp
- heliotik/botglue.nvim — Neovim plugin for AI-assisted inline code editing via the Claude Code CLI: github.com/heliotik/botglue.nvim
- ivan-hilckov/seminar-ai-llm — slides of the one-day seminar “AI and LLM for researchers” at FBU VNIILM, 13 May 2026 (53 slides, Vite + React, deployed on Dokploy): github.com/ivan-hilckov/seminar-ai-llm
- yokemate (private, YokeLoop) — orchestrator chat for development across many organizations and repositories: one Claude Code session, ticket-driven /plan, /do, /review and /ship, shared PostgreSQL queue, PreToolUse guard, 289 tests
- yokemate-cc (private, YokeLoop) — captain-and-crew harness for Claude Code, a fork of kunchenguid/firstmate, “talk to one agent, ship with a crew”: one agent dispatches and supervises a crew in multiplexer windows, disposable git worktrees, ship and scout tasks, event-driven supervision that spends no tokens while idle
- Geospatial platform demo (YouTube): youtu.be/df4GsBd9a_U · gource.io, 4 years of Skyeer: youtu.be/0VSIHanKoNY
- Telegram bot templates: github.com/ivan-hilckov/hello-bot · github.com/ivan-hilckov/hello-ai-bot · github.com/ivan-hilckov/english-teacher-bot

Availability & Preferences

- Immediate availability for Lead AI Engineer, Applied AI Engineer, Forward Deployed Engineer and AI-native engineering lead roles.
- Remote-first; open to hybrid; open to relocation; open to international contract and invoicing.
- Preferred domains: AI-native development platforms and tooling, self-hosted LLM infrastructure, streaming media, geospatial technologies.

Evidence Notes

How the metrics above were measured, so that they can be checked rather than taken on trust:

- Commit counts come from the GitHub search API (author:ivan-hilckov committer-date:2025-06-01..2026-09-17 = 5,712; the two preceding 12-month windows = 80 and 58, counted only on repositories visible to the account today; employer repositories from those years are not included) and from git log over 31 local clones (4,076 non-merge commits, 19,903 file changes, 4 reverts). Both index default branches; local logs include all branches.
- About 1,500 commits made between June and August 2026 carry a mistyped git email (gamil.com) and are not linked to the account, so the public contribution graph and any author: query undercount them; the combined figure of about 7,200 commits adds them back via an author-email: query. Linking the email in GitHub settings would re-attribute them.
- Pull-request figures are is:pr author:ivan-hilckov created: ≥ 2025-06-01 (888), the same with is:merged (828) and reviewed-by:ivan-hilckov (47). Reviews in the own organization happen inside the agent pipeline (ship reports, re-review rounds), not in the GitHub review UI, which is why the reviewed count is low there.
- The Markdown-versus-code split (38% docs-only commits; about 14,000 lines of agent configuration in CLAUDE.md, AGENTS.md and .claude/; about 17,000 lines of skills) is a --numstat bucketing of the local clones after excluding lockfiles, a knowledge base committed in two repositories and a 21,000-line .agents/ template imported into three repositories.
- Test counts are grep counts of test cases in yokemate's test/ directory (289) and a file listing in nook (52 test files). Guard entries are the line count of the orchestrator's guard.jsonl (228 over 2026-09-07 to 09-16: 202 denials, 26

confirmations). Skill, subagent and MCP counts are directory listings and configuration keys on the working machine. Pipeline figures (tickets, stage medians, interventions, cost per ticket) are the output of the orchestrator's own metrics --pipeline command over its journal and Claude Code transcripts for 2026-08-15 to 2026-09-17; cost is understated where a session recorded no cost state, and about half of the tracked spend sits in sessions attributed to no ticket.

- Metrics from roles before 2025 (40% latency reduction, 80% release stability, 700,000 requests per second, $\pm 0.02 \text{ m}^3$) are carried over from the previous version of this resume as reported at the time and were not re-measured for this revision.